Free and Open Source Tools for Research

Jesse Oldroyd

June 15, 2026

2026-06-15

Free and Open Source Tools for Research

Jesse Oldroyd
June 15, 2026

What is FOSS?

What is FOSS?

Definition

Free and open source software (FOSS) is software

- that is freely available (usually under a permissive license);

- has accessible source code.

Free vs. Open

More restrictive definitions exist [1]!

Definition
Free and open source software (FOSS) is software
that is freely available (usually under a permissive license);
has accessible source code.
Free vs. Open
More restrictive definitions exist [1]!

1. Some define free software as the antithesis of proprietary software, whereas open software is treated more permissively.

Definition

Free and open source software (FOSS) is software

- that is freely available (usually under a permissive license);
- has accessible source code.

Free vs. Open

More restrictive definitions exist [1]!

Definition
Free and open source software (FOSS) is software
• that is freely available (usually under a permissive license);
• has accessible source code.
Free vs. Open
More restrictive definitions exist [1]!

1. Some define free software as the antithesis of proprietary software, whereas open software is treated more permissively.

Definition

Free and open source software (FOSS) is software

- that is freely available (usually under a permissive license);
- has accessible source code.

Free vs. Open

More restrictive definitions exist [1]!

Definition
Free and open source software (FOSS) is software
- that is freely available (usually under a permissive license);
- has accessible source code.
Free vs. Open
More restrictive definitions exist [1]!

1. Some define free software as the antithesis of proprietary software, whereas open software is treated more permissively.

Philosophy of Free Software

The philosophy of free software can be encapsulated by *four essential freedoms* [2]:

1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software

The philosophy of free software can be encapsulated by *four essential freedoms* [2]:

1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software

The philosophy of free software can be encapsulated by *four essential freedoms* [2]:

1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software
The philosophy of free software can be encapsulated by *four essential freedoms* [2]:
1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software

The philosophy of free software can be encapsulated by *four essential freedoms* [2]:

1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software
The philosophy of free software can be encapsulated by *four essential freedoms* [2]:
1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software

The philosophy of free software can be encapsulated by *four essential freedoms* [2]:

1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software
The philosophy of free software can be encapsulated by *four essential freedoms* [2]:
1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software

The philosophy of free software can be encapsulated by *four essential freedoms* [2]:

1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Philosophy of Free Software
The philosophy of free software can be encapsulated by *four essential freedoms* [2]:
1. to use the software.
2. to inspect and change the software.
3. to share copies of the original software.
4. to share copies of the modified software.

Licenses

The freedoms associated with the use of free software are given legal weight by various licenses, classified as *copyleft* or *permissive* [3].

- Copyleft licenses: protects against proprietary software development.

- Permissive: encourages adoption and collaboration.

Licenses
The freedoms associated with the use of free software are given legal weight by various licenses, classified as *copyleft* or *permissive* [3].
- Copyleft licenses: protects against proprietary software development.
- Permissive: encourages adoption and collaboration.

1. Copyleft licenses can be used if you're planning to use your software for commercial purposes.
2. Permissive licenses allow for your work to be used in commercial contexts.

Licenses

The freedoms associated with the use of free software are given legal weight by various licenses, classified as *copyleft* or *permissive* [3].

- Copyleft licenses: protects against proprietary software development.

- Permissive: encourages adoption and collaboration.

Licenses
The freedoms associated with the use of free software are given legal weight by various licenses, classified as *copyleft* or *permissive* [3].
- Copyleft licenses: protects against proprietary software development.
- Permissive: encourages adoption and collaboration.

1. Copyleft licenses can be used if you're planning to use your software for commercial purposes.
2. Permissive licenses allow for your work to be used in commercial contexts.

Licenses

The freedoms associated with the use of free software are given legal weight by various licenses, classified as *copyleft* or *permissive* [3].

- Copyleft licenses: protects against proprietary software development.
- Permissive: encourages adoption and collaboration.

Licenses

The freedoms associated with the use of free software are given legal weight by various licenses, classified as *copyleft* or *permissive* [3].

- Copyleft licenses: protects against proprietary software development.
- Permissive: encourages adoption and collaboration.

1. Copyleft licenses can be used if you're planning to use your software for commercial purposes.
2. Permissive licenses allow for your work to be used in commercial contexts.

When FOSS Is the Right Choice

FOSS might be the right choice if you require

- Transparency.
- Composability.
- Customizability.

When FOSS Is the Right Choice
FOSS might be the right choice if you require
- Transparency.
- Composability.
- Customizability.

1. Transparency lets you inspect software itself to verify results. Code that uses proprietary software makes it difficult, if not impossible, to confirm the correctness of results or to identify errors.
2. Free software is often text-based. Since so many use this common format, it can be relatively painless to chain multiple tools together into a single workflow.
3. Given how permissive many open source tools are, it's possible (and often expected) for you to customize the tool to your own needs.

When FOSS Is the Right Choice

FOSS might be the right choice if you require

- Transparency.
- Composability.
- Customizability.

When FOSS Is the Right Choice
FOSS might be the right choice if you require
- Transparency.
- Composability.
- Customizability.

1. Transparency lets you inspect software itself to verify results. Code that uses proprietary software makes it difficult, if not impossible, to confirm the correctness of results or to identify errors.
2. Free software is often text-based. Since so many use this common format, it can be relatively painless to chain multiple tools together into a single workflow.
3. Given how permissive many open source tools are, it's possible (and often expected) for you to customize the tool to your own needs.

When FOSS Is the Right Choice

FOSS might be the right choice if you require

- Transparency.
- Composability.
- Customizability.

When FOSS Is the Right Choice
FOSS might be the right choice if you require
• Transparency.
• Composability.
• Customizability.

1. Transparency lets you inspect software itself to verify results. Code that uses proprietary software makes it difficult, if not impossible, to confirm the correctness of results or to identify errors.
2. Free software is often text-based. Since so many use this common format, it can be relatively painless to chain multiple tools together into a single workflow.
3. Given how permissive many open source tools are, it's possible (and often expected) for you to customize the tool to your own needs.

When FOSS Is the Right Choice

FOSS might be the right choice if you require

- Transparency.
- Composability.
- Customizability.

When FOSS Is the Right Choice
FOSS might be the right choice if you require
• Transparency.
• Composability.
• Customizability.

1. Transparency lets you inspect software itself to verify results. Code that uses proprietary software makes it difficult, if not impossible, to confirm the correctness of results or to identify errors.
2. Free software is often text-based. Since so many use this common format, it can be relatively painless to chain multiple tools together into a single workflow.
3. Given how permissive many open source tools are, it's possible (and often expected) for you to customize the tool to your own needs.

When FOSS Is the Wrong Choice

Despite good reasons to use them, free and open source tools are not always a good idea. You need to be mindful of the following:

- Use tools that work well with what your collaborators are using.
- FOSS tools are often not as polished as their proprietary counterparts.
- Security concerns.

When FOSS Is the Wrong Choice

Despite good reasons to use them, free and open source tools are not always a good idea. You need to be mindful of the following:

- Use tools that work well with what your collaborators are using.
- FOSS tools are often not as polished as their proprietary counterparts.
- Security concerns.

1. In particular, follow any relevant journal guidelines!
2. FOSS tools are also more subject to becoming "abandonware" as maintaining them is time-consuming—and often thankless—work.
3. While many open-source projects are well-maintained, their accessibility to the public does leave them at risk of being used by malicious actors. You should only use tools from trustworthy sources!

When FOSS Is the Wrong Choice

Despite good reasons to use them, free and open source tools are not always a good idea. You need to be mindful of the following:

- Use tools that work well with what your collaborators are using.
- FOSS tools are often not as polished as their proprietary counterparts.
- Security concerns.

When FOSS Is the Wrong Choice
Despite good reasons to use them, free and open source tools are not always a good idea. You need to be mindful of the following:
- Use tools that work well with what your collaborators are using.
- FOSS tools are often not as polished as their proprietary counterparts.
- Security concerns.

1. In particular, follow any relevant journal guidelines!
2. FOSS tools are also more subject to becoming “abandonware” as maintaining them is time-consuming—and often thankless—work.
3. While many open-source projects are well-maintained, their accessibility to the public does leave them at risk of being used by malicious actors. You should only use tools from trustworthy sources!

When FOSS Is the Wrong Choice

Despite good reasons to use them, free and open source tools are not always a good idea. You need to be mindful of the following:

- Use tools that work well with what your collaborators are using.
- FOSS tools are often not as polished as their proprietary counterparts.
- Security concerns.

When FOSS Is the Wrong Choice
Despite good reasons to use them, free and open source tools are not always a good idea. You need to be mindful of the following:
- Use tools that work well with what your collaborators are using.
- FOSS tools are often not as polished as their proprietary counterparts.
- Security concerns.

1. In particular, follow any relevant journal guidelines!
2. FOSS tools are also more subject to becoming “abandonware” as maintaining them is time-consuming—and often thankless—work.
3. While many open-source projects are well-maintained, their accessibility to the public does leave them at risk of being used by malicious actors. You should only use tools from trustworthy sources!

When FOSS Is the Wrong Choice

Despite good reasons to use them, free and open source tools are not always a good idea. You need to be mindful of the following:

- Use tools that work well with what your collaborators are using.
- FOSS tools are often not as polished as their proprietary counterparts.
- Security concerns.

When FOSS Is the Wrong Choice
Despite good reasons to use them, free and open source tools are not always a good idea. You need to be mindful of the following:
- Use tools that work well with what your collaborators are using.
- FOSS tools are often not as polished as their proprietary counterparts.
- Security concerns.

1. In particular, follow any relevant journal guidelines!
2. FOSS tools are also more subject to becoming "abandonware" as maintaining them is time-consuming—and often thankless—work.
3. While many open-source projects are well-maintained, their accessibility to the public does leave them at risk of being used by malicious actors. You should only use tools from trustworthy sources!

Tools

Tools

Types of Tools

- There is a large selection of open source tools available, including coding and writing.

- One of the most fundamental tools is a text editor.

- Even though you start with text, output is not necessarily just a text file!

Types of Tools
- There is a large selection of open source tools available, including coding and writing.
- One of the most fundamental tools is a text editor.
- Even though you start with text, output is not necessarily just a text file!

Types of Tools

- There is a large selection of open source tools available, including coding and writing.
- One of the most fundamental tools is a text editor.
- Even though you start with text, output is not necessarily just a text file!

Types of Tools

- There is a large selection of open source tools available, including coding and writing.
- One of the most fundamental tools is a text editor.
- Even though you start with text, output is not necessarily just a text file!

Types of Tools

- There is a large selection of open source tools available, including coding and writing.
- One of the most fundamental tools is a text editor.
- Even though you start with text, output is not necessarily just a text file!

2026-06-15

Free and Open Source Tools for Research
└FOSS Tools
└Types of Tools

Types of Tools

- There is a large selection of open source tools available, including coding and writing.
- One of the most fundamental tools is a text editor.
- Even though you start with text, output is not necessarily just a text file!

Tools for Computation

- R
- Python with pandas, Seaborn and SciPy
- Python with NumPy
- Python with SymPy

2026-06-15

Free and Open Source Tools for Research
└FOSS Tools
└Coding
└Tools for Computation

Tools for Computation

- R
- Python with pandas, Seaborn and SciPy
- Python with NumPy
- Python with SymPy

Tools for Computation

- R
- Python with pandas, Seaborn and SciPy
- Python with NumPy
- Python with SymPy

Tools for Computation
- R
- Python with pandas, Seaborn and SciPy
- Python with NumPy
- Python with SymPy

Tools for Computation

- R
- Python with pandas, Seaborn and SciPy
- Python with NumPy
- Python with SymPy

2026-06-15

Free and Open Source Tools for Research
└─FOSS Tools
  └─Coding
    └─Tools for Computation

Tools for Computation
- R
- Python with pandas, Seaborn and SciPy
- Python with NumPy
- Python with SymPy

Tools for Computation

- R
- Python with pandas, Seaborn and SciPy
- Python with NumPy
- Python with SymPy

Tools for Computation
- R
- Python with pandas, Seaborn and SciPy
- Python with NumPy
- Python with SymPy

Python Code for Statistics

```python
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_theme(style="dark")

# Simulate data from a bivariate Gaussian
n = 10000
mean = [0, 0]
cov = [(2, .4), (.4, .2)]
rng = np.random.RandomState(0)
x, y = rng.multivariate_normal(mean, cov, n).T

# Draw a combo histogram and scatterplot with density contours
f, ax = plt.subplots(figsize=(6, 6))
sns.scatterplot(x=x, y=y, s=5, color=".15")
sns.histplot(x=x, y=y, bins=50, pthresh=.1, cmap="mako")
sns.kdeplot(x=x, y=y, levels=5, color="w", linewidths=1)
```

Listing 1: Example Python code for statistical visualization

Python Code for Statistics

```python
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_theme(style="dark")

# Simulate data from a bivariate Gaussian
n = 10000
mean = [0, 0]
cov = [(2, .4), (.4, .2)]
rng = np.random.RandomState(0)
x, y = rng.multivariate_normal(mean, cov, n).T

# Draw a combo histogram and scatterplot with density contours
f, ax = plt.subplots(figsize=(6, 6))
sns.scatterplot(x=x, y=y, s=5, color=".15")
sns.histplot(x=x, y=y, bins=50, pthresh=.1, cmap="mako")
sns.kdeplot(x=x, y=y, levels=5, color="w", linewidths=1)
```

Listing 1: Example Python code for statistical visualization

Visualization

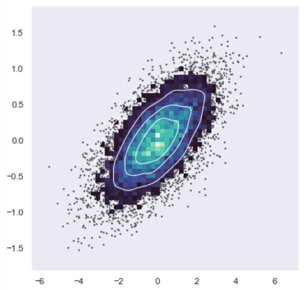
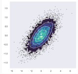


Figure 1: Data visualization produced by 1



IDE vs. Text Editor

- When using programming languages, you will need a program to interact with the language.

- There is often a choice between an *integrated development environment* (IDE) and a text editor.

- Open source text editors: VSCode (new school), Vi(m) and Emacs (*very* old school).

General or specific language support

IDEs are tailored to specific languages (such as RStudio). Text editors tend to be more multi-purpose.

IDE vs. Text Editor
When using programming languages, you will need a program to interact with the language.
There is often a choice between an *integrated development environment* (IDE) and a text editor.
Open source text editors: VSCode (new school), Vi(m) and Emacs (very old school).
General or specific language support
IDEs are tailored to specific languages (such as RStudio). Text editors tend to be more multi-purpose.

IDE vs. Text Editor

- When using programming languages, you will need a program to interact with the language.
- There is often a choice between an *integrated development environment* (IDE) and a text editor.
- Open source text editors: VSCode (new school), Vi(m) and Emacs (*very* old school).

General or specific language support

IDEs are tailored to specific languages (such as RStudio). Text editors tend to be more multi-purpose.

IDE vs. Text Editor
- When using programming languages, you will need a program to interact with the language.
- There is often a choice between an *integrated development environment* (IDE) and a text editor.
- Open source text editors: VSCode (new school), Vi(m) and Emacs (very old school).
General or specific language support
IDEs are tailored to specific languages (such as RStudio). Text editors tend to be more multi-purpose.

IDE vs. Text Editor

- When using programming languages, you will need a program to interact with the language.
- There is often a choice between an *integrated development environment* (IDE) and a text editor.
- Open source text editors: VSCode (new school), Vi(m) and Emacs (*very* old school).

General or specific language support

IDEs are tailored to specific languages (such as RStudio). Text editors tend to be more multi-purpose.

IDE vs. Text Editor

- When using programming languages, you will need a program to interact with the language.
- There is often a choice between an *integrated development environment* (IDE) and a text editor.
- Open source text editors: VSCode (new school), Vi(m) and Emacs (very old school).

General or specific language support

IDEs are tailored to specific languages (such as RStudio). Text editors tend to be more multi-purpose.

IDE vs. Text Editor

- When using programming languages, you will need a program to interact with the language.
- There is often a choice between an *integrated development environment* (IDE) and a text editor.
- Open source text editors: VSCode (new school), Vi(m) and Emacs (*very* old school).

General or specific language support

IDEs are tailored to specific languages (such as RStudio). Text editors tend to be more multi-purpose.

IDE vs. Text Editor
- When using programming languages, you will need a program to interact with the language.
- There is often a choice between an *integrated development environment* (IDE) and a text editor.
- Open source text editors: VSCode (new school), Vi(m) and Emacs (very old school).
General or specific language support
IDEs are tailored to specific languages (such as RStudio). Text editors tend to be more multi-purpose.

Tools for Writing

- LaTeX
- git
- harper-ls and LanguageTool

Tools for Writing
- LaTeX
- git
- harper-ls and LanguageTool

1. Perhaps the most popular open-source tool for creating technical documents out there. Primarily used in mathematics and physics, but there are also packages available for chemistry, biology and more.
2. A tool for managing different versions of a document. This is useful for tracking changes in any text-based document.
3. Tools for checking grammar and spelling in text files.

Tools for Writing

- LaTeX
- git
- harper-ls and LanguageTool

Tools for Writing
- LaTeX
- git
- harper-ls and LanguageTool

1. Perhaps the most popular open-source tool for creating technical documents out there. Primarily used in mathematics and physics, but there are also packages available for chemistry, biology and more.
2. A tool for managing different versions of a document. This is useful for tracking changes in any text-based document.
3. Tools for checking grammar and spelling in text files.

Tools for Writing

- LaTeX
- git
- harper-ls and LanguageTool

Tools for Writing
- LaTeX
- git
- harper-ls and LanguageTool

1. Perhaps the most popular open-source tool for creating technical documents out there. Primarily used in mathematics and physics, but there are also packages available for chemistry, biology and more.
2. A tool for managing different versions of a document. This is useful for tracking changes in any text-based document.
3. Tools for checking grammar and spelling in text files.

Tools for Writing

- $\LaTeX$
- git
- harper-ls and LanguageTool

Tools for Writing
- LaTeX
- git
- harper-ls and LanguageTool

1. Perhaps the most popular open-source tool for creating technical documents out there. Primarily used in mathematics and physics, but there are also packages available for chemistry, biology and more.
2. A tool for managing different versions of a document. This is useful for tracking changes in any text-based document.
3. Tools for checking grammar and spelling in text files.

Tools for Drawing

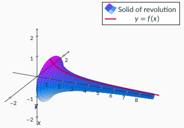
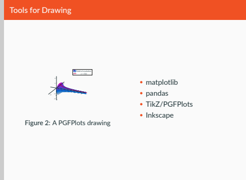


Figure 2: A PGFPlots drawing

- matplotlib
- pandas
- TikZ/PGFPlots
- Inkscape



Composability Again

- The tools mentioned so far are not mutually exclusive!

- These tools have specific purposes, but can be chained together to form a powerful workflow.

Solving a Math Problem

The expanded form of $(x-3)^5$ is ...

Composability Again
- The tools mentioned so far are not mutually exclusive!
- These tools have specific purposes, but can be chained together to form a powerful workflow.

Composability Again

- The tools mentioned so far are not mutually exclusive!

- These tools have specific purposes, but can be chained together to form a powerful workflow.

Solving a Math Problem

The expanded form of $(x-3)^5$ is …

Composability Again
- The tools mentioned so far are not mutually exclusive!
- These tools have specific purposes, but can be chained together to form a powerful workflow.
Solving a Math Problem
The expanded form of $(x-3)^5$ is …

Composability Again

- The tools mentioned so far are not mutually exclusive!

- These tools have specific purposes, but can be chained together to form a powerful workflow.

Solving a Math Problem

The expanded form of $(x-3)^5$ is …

Composability Again
- The tools mentioned so far are not mutually exclusive!
- These tools have specific purposes, but can be chained together to form a powerful workflow.
Solving a Math Problem
The expanded form of $(x-3)^5$ is …

Sharing Your Work

Platforms for distributing work:

- GitHub

- personal website.

Dangers of Sharing Work

You should be careful when sharing potentially sensitive work. Always check with your advisor and collaborators!

Sharing Your Work
Platforms for distributing work:
• GitHub
• personal website.
Dangers of Sharing Work
You should be careful when sharing potentially sensitive work. Always check with your advisor and collaborators!

1. GitHub is owned by Microsoft, but allows some free storage. Beware of AI training!

Sharing Your Work

Platforms for distributing work:

- GitHub
- personal website.

Dangers of Sharing Work

You should be careful when sharing potentially sensitive work. Always check with your advisor and collaborators!

Sharing Your Work
Platforms for distributing work:
- GitHub
- personal website.
Dangers of Sharing Work
You should be careful when sharing potentially sensitive work. Always check with your advisor and collaborators!

1. GitHub is owned by Microsoft, but allows some free storage. Beware of AI training!

Sharing Your Work

Platforms for distributing work:

- GitHub
- personal website.

Dangers of Sharing Work

You should be careful when sharing potentially sensitive work. Always check with your advisor and collaborators!

Sharing Your Work
Platforms for distributing work:
• GitHub
• personal website.
Dangers of Sharing Work
You should be careful when sharing potentially sensitive work. Always check with your advisor and collaborators!

1. GitHub is owned by Microsoft, but allows some free storage. Beware of AI training!

Literate Programming

- Work or results that depend on computer analysis can be shared using Jupyter Notebooks.

- Sharing code and a narrative is often more instructive than just sharing the code itself. See: gravity wave notebooks.

Literate Programming

- Work or results that depend on computer analysis can be shared using Jupyter Notebooks.
- Sharing code and a narrative is often more instructive than just sharing the code itself. See: gravity wave notebooks.

1. Other tools include Quarto and Typst.
2. Notebooks can also be hosted via Google Colab for interactivity.

Literate Programming

- Work or results that depend on computer analysis can be shared using Jupyter Notebooks.

- Sharing code and a narrative is often more instructive than just sharing the code itself. See: gravity wave notebooks.

Literate Programming

- Work or results that depend on computer analysis can be shared using Jupyter Notebooks.
- Sharing code and a narrative is often more instructive than just sharing the code itself. See: gravity wave notebooks.

1. Other tools include Quarto and Typst.
2. Notebooks can also be hosted via Google Colab for interactivity.

Literate Programming

- Work or results that depend on computer analysis can be shared using Jupyter Notebooks.

- Sharing code and a narrative is often more instructive than just sharing the code itself. See: gravity wave notebooks.

2026-06-15

Literate Programming

- Work or results that depend on computer analysis can be shared using Jupyter Notebooks.

- Sharing code and a narrative is often more instructive than just sharing the code itself. See: gravity wave notebooks.

1. Other tools include Quarto and Typst.
2. Notebooks can also be hosted via Google Colab for interactivity.

Licensing Your Work

If you are going to share your work, you should specify a license that outlines the following:

- how your work can be modified and shared.
- in what contexts your work can be used.

Choosing a License

To walk you through the process of choosing a license, you can use tools such as https://choosealicense.com/.

Licensing Your Work
If you are going to share your work, you should specify a license that outlines the following:
how your work can be modified and shared.
in what contexts your work can be used.
Choosing a License
To walk you through the process of choosing a license, you can use tools such as https://choosealicense.com/.

Licensing Your Work

If you are going to share your work, you should specify a license that outlines the following:

- how your work can be modified and shared.

- in what contexts your work can be used.

Choosing a License

To walk you through the process of choosing a license, you can use tools such as https://choosealicense.com/.

Licensing Your Work
If you are going to share your work, you should specify a license that outlines the following:
• how your work can be modified and shared.
• in what contexts your work can be used.
Choosing a License
To walk you through the process of choosing a license, you can use tools such as https://choosealicense.com/.

1. GitHub will actually prompt you to provide a license.

Licensing Your Work

If you are going to share your work, you should specify a license that outlines the following:

- how your work can be modified and shared.
- in what contexts your work can be used.

Choosing a License

To walk you through the process of choosing a license, you can use tools such as https://choosealicense.com/.

Licensing Your Work
If you are going to share your work, you should specify a license that outlines the following:
• how your work can be modified and shared.
• in what contexts your work can be used.
Choosing a License
To walk you through the process of choosing a license, you can use tools such as https://choosealicense.com/.

1. GitHub will actually prompt you to provide a license.
2. In particular, can it be used in proprietary software?

Licensing Your Work

If you are going to share your work, you should specify a license that outlines the following:

- how your work can be modified and shared.
- in what contexts your work can be used.

Choosing a License

To walk you through the process of choosing a license, you can use tools such as https://choosealicense.com/.

Licensing Your Work

If you are going to share your work, you should specify a license that outlines the following:

- how your work can be modified and shared.
- in what contexts your work can be used.

Choosing a License

To walk you through the process of choosing a license, you can use tools such as https://choosealicense.com/.

1. GitHub will actually prompt you to provide a license.
2. In particular, can it be used in proprietary software?

Conclusion

Conclusion

Conclusion

- Open source tools can form the basis of a powerful workflow.

- Despite their utility, these tools often take longer to learn and are not as immediately intuitive as their proprietary counterparts.

- You need to decide if they are appropriate for your project!

Conclusion

- Open source tools can form the basis of a powerful workflow.
- Despite their utility, these tools often take longer to learn and are not as immediately intuitive as their proprietary counterparts.
- You need to decide if they are appropriate for your project!

Conclusion

- Open source tools can form the basis of a powerful workflow.
- Despite their utility, these tools often take longer to learn and are not as immediately intuitive as their proprietary counterparts.
- You need to decide if they are appropriate for your project!

Conclusion
- Open source tools can form the basis of a powerful workflow.
- Despite their utility, these tools often take longer to learn and are not as immediately intuitive as their proprietary counterparts.
- You need to decide if they are appropriate for your project!

Conclusion

- Open source tools can form the basis of a powerful workflow.
- Despite their utility, these tools often take longer to learn and are not as immediately intuitive as their proprietary counterparts.
- You need to decide if they are appropriate for your project!

Conclusion
- Open source tools can form the basis of a powerful workflow.
- Despite their utility, these tools often take longer to learn and are not as immediately intuitive as their proprietary counterparts.
- You need to decide if they are appropriate for your project!

References

[1] *Open Source - GNU Project - Free Software Foundation (FSF)*. July 3, 2007. URL: https://web.archive.org/web/20070703091340/https://www.gnu.org.ua/philosophy/free-software-for-freedom.html (visited on 06/14/2026).

[2] *Philosophy of the GNU Project - GNU Project - Free Software Foundation*. URL: https://www.gnu.org/philosophy/philosophy.html (visited on 06/14/2026).

[3] *Choose a License – Open Source Programs Office*. URL: https://ospo.library.jhu.edu/learn-grow/licensing-overview/choose-a-license/ (visited on 06/14/2026).

References

[1] Open Source - GNU Project - Free Software Foundation (FSF). July 3, 2007. URL: https://web.archive.org/web/20070703091340/https://www.gnu.org.ua/philosophy/free-software-for-freedom.html (visited on 06/14/2026).

[2] Philosophy of the GNU Project - GNU Project - Free Software Foundation. URL: https://www.gnu.org/philosophy/philosophy.html (visited on 06/14/2026).

[3] Choose a License – Open Source Programs Office. URL: https://ospo.library.jhu.edu/learn-grow/licensing-overview/choose-a-license/ (visited on 06/14/2026).